

## Web Developer & Digital Infrastructure Specialist| *Talent Acquisition Process*

### Assessment - Version 1

---

#### Objective

This assessment is designed to evaluate your ability to build, troubleshoot, analyze, and optimize modern websites in a fast-paced digital marketing agency environment. We aim to understand your proficiency in front-end development, design implementation, tracking and integrations, website performance, hosting fundamentals, and overall technical decision-making.

#### Submission Guidelines

- **Format:** Google Docs, Notion, or PDF and any supporting files.
- **Deadline:** Jun 29, 2026 7:00 AM

#### Evaluation Criteria

- Website Development & Figma Interpretation (30%)
- Tracking, Integrations & Technical Problem Solving (25%)
- Technical and Performance Audit (20%)
- Custom FAQ Accordion with Live Search Filter (15%)
- Hosting, Security & Infrastructure Knowledge (15%)

#### Scenario Overview

White Shark Media has acquired a new client that requires a landing page launch on a tight timeline. The Sales, SEO, and Operations teams depend on you to ensure the page launches correctly, tracks leads accurately, loads quickly, and follows best practices. You have been asked to review the following requirements and provide your recommendations and implementation plan.

#### Task 1: Figma to Web Execution

#### Task #2 - Lead Tracking & Integrations

#### Task #3 - Custom FAQ Accordion with Live Search Filter

#### Task #4 - Hosting & Security Incident

#### Task 5: Technical and Performance Audit (White Shark Media)

## Web Developer & Digital Infrastructure Specialist

**Candidate:** Fernando José Obando Velásquez

**Date:** Jun/29/2026

### Summary:

This document contains the strategic analysis and technical execution for the White Shark Media assessment. The development tasks (1 and 3) were built using a self-contained, lightweight architecture prioritizing performance and SEO. The analytical tasks include diagnostic evaluations and actionable recommendations for tracking, performance, and infrastructure management.

### Task 1: Figma to Web Execution

#### Deliverables:

- **Live Staging Environment:** [Crypgo](#)
- **Source Code (Repository):** [Github](#)

#### Technical Approach & Decisions:

- **Architecture:** Built with Semantic HTML5 and pure CSS3 (Flexbox) without heavy frameworks to ensure optimal Core Web Vitals.
- **Responsiveness:** Implemented a mobile-first approach. The navigation bar utilizes a custom absolute-positioning strategy to prevent document reflow on mobile devices.
- **Performance:** Assets were optimized, and no external libraries were used, guaranteeing instant load times.

### Task 2: Lead Tracking & Integrations

**Diagnostics & Systems Reviewed:** To investigate intermittent lead loss, the troubleshooting must follow the data journey, isolating the endpoints before analyzing the middleware. The systems reviewed include:

- **The Origin (Frontend):** The browser's Network tab to verify if the form payload is successfully dispatched upon submission.
- **The Middleware (Integration):** Tag management systems (e.g., GTM), Webhooks, or automation tools (e.g., Zapier) task history to check for timeouts or failed executions.
- **The Destination (CRM):** The CRM's API error logs or import history to check for rejected payloads.

#### Potential Causes of Intermittent Lead Loss

- **Client-Side Blocking:** Aggressive privacy browsers (e.g., Brave) or Intelligent Tracking Prevention (ITP) blocking third-party tracking scripts, preventing the data from leaving the browser.

- **Payload Variations:** Forms containing heavy file attachments that cause timeouts during the webhook transmission.
- **CRM Validation Rules:** Strict deduplication rules, mismatched mandatory fields, or data-type formatting issues (e.g., date formats) causing the CRM to silently reject specific leads.

**Stakeholder Communication Strategy** Findings will be communicated by translating technical issues into business impact. Instead of discussing "API payloads or HTTP errors," the focus will be on what is being lost (e.g., "The system is discarding leads who leave the phone number empty"), why it happens ("A mismatch in mandatory rules between the website and the CRM"), and the *exact timeline* for the deployed fix.

### Deliverable: Step-by-Step Troubleshooting Plan

1. **Replicate & Isolate:** Submit controlled test forms mimicking different user behaviors (e.g., with/without attachments, different browsers, edge-case data).
2. **Inspect Origin (Browser):** Monitor the browser's Network tab to confirm the **POST** request returns a **200 OK** status and the payload is correctly formatted.
3. **Audit Middleware:** Review the webhook/automation history logs to verify the data was received and forwarded without timeouts.
4. **Audit Destination (CRM):** Cross-reference missing leads with the CRM's API error logs to identify rejection codes (e.g., **400 Bad Requests** due to validation failures).
5. **Resolve & Monitor:** Align frontend validation rules with CRM database requirements and monitor live traffic for 48 hours to ensure consistent data flow.

### Task 3: Custom FAQ Accordion with Live Search Filter

#### Deliverables:

- **Self-Contained File:** [FAQs view](#)
- **RAW File:** [Github](#)
- **Code Walkthrough Video:** [Spanish version](#) [English version](#)

#### Technical Approach & Decisions:

- **Live Search Logic:** Utilized Vanilla JS event listeners (**input**) to read the **innerText** of FAQ blocks, forcing lowercase comparison for case-insensitive, real-time DOM filtering.
- **Accordion Logic:** Engineered a "clean-slate" approach using **forEach** to remove active classes from all items before opening the targeted question, ensuring only one answer displays at a time.
- **Smooth Animations:** Avoided animating the **display** property. Instead, utilized CSS transitions on **max-height** for a native, smooth reveal effect.

## Task 4: Hosting & Security Incident

### Immediate Response Process (Containment)

1. **Isolate the Threat:** Immediately route all traffic to a static, secure "Maintenance Mode" page (HTTP 503 status). This protects end-users from malicious payloads and preserves SEO rankings during the downtime.
2. **Revoke Access:** Force a global password reset and invalidate all active session tokens for hosting control panels, CMS admins, and FTP/SSH accounts to lock the attacker out.

### Investigation & Remediation Steps

1. **Version Control Audit:** Inspect the CI/CD pipeline and Git commit history for unauthorized pushes or modified files.
2. **Rollback Strategy:** Execute a hard rollback to the last known secure deployment (Known Good State).
3. **Sandbox Analysis:** Download the compromised environment into an isolated local sandbox to identify the entry vector (e.g., outdated plugin, compromised credentials, or SQL injection).
4. **SSL/TLS Verification:** Reissue and force HTTPS, as attackers often downgrade security protocols to trigger browser warnings.

## Task 5: Technical and Performance Audit ([www.whitesharkmedia.com](http://www.whitesharkmedia.com))

PageSpeed Insight: [Results](#)

Wappalyzer: [Results](#)

UI Evidence: [Github](#)

**Summary** A comprehensive 360-degree audit of the live site—spanning Performance, Infrastructure, and UI/UX—reveals a critical inversion of typical metrics: the Desktop environment (37/100) severely underperforms compared to Mobile (51/100). An architectural analysis (via Wappalyzer and Chrome DevTools Network Profiling) exposes a monolithic WordPress environment saturated with global plugins and third-party trackers. This bloat chokes the browser's main thread, forces sequential network request chaining, and directly causes critical responsive layout breaks and navigation state failures across standard viewports (Mobile, iPad, Desktop).

### Key Findings & Business Impact

- **High Impact (Network Waterfall Chaining & Main Thread Blocking):** DevTools Network profiling exposes severe sequential loading (request chaining) rather than parallelized fetching, pushing the **DOMContentLoaded** and **Load** events dangerously close to the 3-second mark. Combined with **15.5s** of main-thread work and **9.4s** of JS execution, this creates a severe "hydration delay."

- **Business Impact:** On mobile devices, the global hamburger menu takes several seconds to become interactive after the visual paint. This severe input delay causes immediate user frustration and increased bounce rates before any lead-gen action can occur.
- **High Impact (UI/UX Z-Index Collisions & Responsive Breaks):** DevTools inspection reveals critical CSS management failures. The absolute positioning of the promotional top banner fails to push the DOM content down, causing it to overlap and obscure the company logo and global navbar across multiple breakpoints (observed on iPhone 14 Pro and iPad Pro).
  - **Business Impact:** Obscuring the primary brand logo and navigation links looks highly unprofessional and breaks the fundamental user journey.
- **Medium Impact (DOM State Management Failure):** The JavaScript handling the scroll-triggered navbar conflicts with the mobile menu's overlay. Scrolling up while the menu is active breaks the state machine, trapping the user behind an un-clickable invisible overlay.
- **Medium Impact (SEO & Accessibility):** The `robots.txt` file returns an invalid syntax error. Additionally, primary Lead-Gen forms lack associated `<label>` tags, and the viewport actively blocks user zooming (`user-scalable="no"`), violating basic WCAG standards.

## Immediate Actionable Recommendations

1. **Refactor Global CSS & Z-Index Stack:** Immediately audit the CSS architecture for the global header. Replace arbitrary absolute positioning with semantic Flexbox/Grid layouts to ensure the promotional banner properly displaces the navbar without overlapping.
2. **Debug Navigation State JS & Network Chaining:** Refactor the scroll-event listeners to prevent the mobile UI trap. Implement strict `defer` and `async` attributes on third-party tags to break the sequential loading chain and free up the main thread.
3. **SEO & A11Y Remediation:** Correct the `robots.txt` syntax, inject `<label>` tags into all form inputs, and remove anti-zoom viewport restrictions.

## Strategic Bonus: Architecture Modernization (The Headless Approach)

While immediate CSS/JS fixes address the visual symptoms, the root cause of the 15.5s main-thread blocking and network waterfall chaining is the structural limitation of the current monolithic WordPress architecture. To achieve a 100/100 Core Web Vitals score without disrupting the marketing team, I recommend migrating the frontend to a [Headless Architecture utilizing Astro](#).

- **The Technical Cure:** Astro's "Islands Architecture" ships zero JavaScript by default. This instantly eliminates the 9.4s JS penalty, ensuring the mobile menu is interactive at millisecond zero. Furthermore, offloading marketing trackers to Web Workers (via tools like Partytown) will protect the main thread and restore parallel network fetching.

- **The Business Value:** The SEO and Content teams retain their exact current backend (WordPress, ACF), while the Astro frontend compiles into lightning-fast, static pages deployed on an edge network, guaranteeing a flawless, app-like UI across all viewports.

#### AI-Assisted Observations Disclosure

- **Tool Used:** Google Gemini.
- **Scope of Use:** AI was utilized to analyze raw diagnostic data (PageSpeed Insights, Wappalyzer, Network Profiling), translate DevTools viewport anomalies into technical architectural flaws, and structure this executive report.